Syndromes Solved Directly: A Zero-Signing-Query Forgery against SSD

Martin Feussner
Selmer Center, University of Bergen
martin.feussner@uib.no

14 October 2026

This audit used the OpenAI model Daybreak Blue at the ultra reasoning level. OpenAI Codex generated and selected this paper's title and intentionally chose "Syndromes Solved Directly" as wordplay that spells the scheme acronym SSD and describes the direct public linear solve used by the forgery. OpenAI Codex performed AI-assisted cryptanalysis, implementation, experimentation, adversarial-review coordination, and technical writing. Primary reproduction, a separately tasked hostile adversarial review, fresh-context independent reproduction, the final exact-target attack-history recheck, and independent Publication Reviewer review passed. No independent human verification was performed or required.

Abstract

We give a public-key-only, zero-signing-query existential fresh-message forgery against standard-mode SSD1760 from Abdelhaliem Babiker's *Digital Signature from Syndrome Decoding Problem*, IACR ePrint 2022/1698. The ordinary verifier accepts $s \in \mathbb{F}_2^{1760}$ when

$$473 \le \mathsf{wt}(s) \le 513 \quad \text{and} \quad Hs = Y\bar{h}.$$

A valid 242×1760 public matrix H has 242 pivots and 1518 free coordinates. The attacker fixes 363 free coordinates to one and directly solves the pivots for each public message target. In the paper's random-oracle model, full-row-rank Y makes the pivot vector uniform in $\mathbb{F}_2^{242}$. Acceptance therefore occurs with exact probability

$$\Pr[110 \le \mathrm{Bin}(242, 1/2) \le 150] = 0.9303737782253141$$

per distinct message, giving 1.0748368273 expected public hash queries and no signing query. This is an existential message-search attack, not a universal or fixed-preselected-message forgery.

The primary Python and hostile Magma reconstructions generated valid structured keys, accepted honest signatures, and produced ordinary-verifier forgeries on their first candidate messages. A fresh-context independent Python reproduction then forged successfully for five of five independently generated full-rank keys, using one or two candidate messages per key and zero signing queries. The clean adversarial attack stage took 0.19 CPU seconds from before row reduction; the independent attacks took 0.052984–0.053914 seconds per key from the same boundary. Changed-message, changed-response, wrong-key, and equation-valid weight-boundary controls behaved as required. The claim is limited to valid full-rank standard SSD1760 keys in the random-oracle model. LVK mode, other parameter rows, every fixed message, and an unconditional SHAKE256 claim are outside scope.

1 Introduction

Signature from Syndrome Decoding (SSD) presents a binary hash-and-sign design whose ordinary verifier checks a public linear equation and a Hamming-weight interval [1]. The proposal argues that finding a response in this interval requires solving a hard syndrome-decoding instance. For its first advertised row, however, the public matrix has only 242 rows and the accepted weight interval is centered broadly around half of those pivot coordinates after a suitable public affine choice.

The attack uses standard Gaussian elimination. Put the public matrix in systematic form, choose a fixed assignment to its free coordinates, and solve the pivots for the public message syndrome. The

resulting vector always satisfies the equation. Its only remaining condition is weight. Fresh random-oracle messages randomize the pivot assignment, and the advertised interval accepts more than 93 percent of those assignments.

This distinction matters for the security claim. The attacker does not control the random-oracle target and cannot promise success for every fixed message. Instead, it chooses distinct fresh messages, computes a public completion for each, and retains the first accepted pair. Existential unforgeability permits the attacker to choose its forged message [2], so this constant-probability public search is sufficient. No message is submitted to a signing oracle.

Contribution and provenance. We claim no novelty for Gaussian elimination, systematic form, affine linear solving, or binomial probabilities. The result is their concrete application to standard SSD1760's ordinary verifier at complete parameters. Primary reproduction and a hostile adversarial review passed. Fresh-context independent reproduction, final exact-target history review, and independent publication review also passed. We make no priority claim and no technique novelty claim.

2 Target and exact scope

2.1 Pinned paper

The target is Abdelhaliem Babiker's *Digital Signature from Syndrome Decoding Problem*, IACR ePrint 2022/1698 [1]. The pinned current object is a 458,702-byte, 14-page PDF with SHA-256 digest

`619903383e40bed6320d1fc0a17c61553073d11545eb12302cafe6a84cfbd80e.`

Its text extraction has SHA-256 digest

`f2962618bf752d315aac0759e286491e6afffccf6d5764bd2c4c89f654c4a62b.`

The official record observed on 14 October 2026 listed receipt on 7 December 2022 and approval on 10 December 2022. The version-list endpoint could not be enumerated, so every claim is pinned to the bytes above.

No author implementation, concrete hash instantiation, message encoding, or signature serialization was available. The executable evidence uses independent paper-level reconstructions and an explicit SHAKE256 adapter. The security theorem below uses the paper's stated random-oracle model and does not depend on SHAKE256 behavior at every fixed message.

2.2 Standard SSD1760 verifier

The claimed row is standard SSD1760:

$$(n, k', d, \omega, \mu, k) = (1760, 242, 502, 484, 11, 440), \qquad \gamma = 4.$$

The public matrices are

$$H \in \mathbb{F}_2^{242\times 1760}, \qquad Y \in \mathbb{F}_2^{242\times 440}.$$

The hash $h \in \mathbb{F}_2^{1760}$ is parsed into four 440-bit blocks $h_1, \ldots, h_4$, and

$$\bar{h} = h_1 + h_2 + h_3 + h_4 \in \mathbb{F}_2^{440}.$$

Algorithm 4 accepts a response $s \in \mathbb{F}_2^{1760}$ precisely when

$$473 \le \mathsf{wt}(s) \le 513, \qquad Hs = Y\bar{h}. \tag{1}$$

There is no further distribution, support, proof, or encoding check in that ordinary verifier.

The result is limited to valid public keys satisfying $\operatorname{rank}(H) = \operatorname{rank}(Y) = 242$, as the paper's key generation requires. It covers standard mode only. LVK uses a secret subset of public rows and is outside the attack model. SSD2744, SSD3728, malformed keys, and rank-deficient Gen1 draws are also outside the claim.

3 Public affine-completion forgery

3.1 Systematic form

Public Gaussian elimination selects 242 pivot columns of H and leaves $1760 - 242 = 1518$ nonpivot coordinates. Write the corresponding decomposition as

$$H_P s_P + H_F s_F = t, \qquad t = Y\bar{h}.$$

The square pivot matrix H_P is invertible. For any chosen free assignment s_F and target t, the unique pivot assignment is

$$s_P = H_P^{-1}(t + H_F s_F). \tag{2}$$

Thus every public target has an efficiently computable response satisfying the verifier equation. The attack fixes an s_F of weight 363.

Lemma 1 (Uniform public target)**.** *In the paper's random-oracle model, if Y has rank 242, then $Y\bar{h}$ is uniform in $\mathbb{F}_2^{242}$ for every fresh distinct message.*

Proof. The xor map from four 440-bit blocks to one 440-bit vector is surjective, so it maps a uniform 1760-bit random-oracle output to a uniform element of $\mathbb{F}_2^{440}$. A rank-242 linear map $Y : \mathbb{F}_2^{440} \to \mathbb{F}_2^{242}$ is surjective and maps that uniform input to a uniform 242-bit output. Distinct random-oracle queries give independent outputs. □

Theorem 1 (Existential fresh-message forgery)**.** *For a valid full-rank standard SSD1760 public key, the public affine-completion algorithm succeeds on each fresh message with probability*

$$p = \Pr[110 \le \mathrm{Bin}(242, 1/2) \le 150] = 0.9303737782253141.$$

It uses zero signing queries and has expected public message-query count $1/p = 1.0748368273$.

Proof. Equation (2) is an invertible affine map from the uniform target t to the pivot vector s_P. Hence s_P is uniform in $\mathbb{F}_2^{242}$, and its Hamming weight has distribution $\mathrm{Bin}(242, 1/2)$. Pivot and free coordinates are disjoint, so

$$\mathsf{wt}(s) = 363 + \mathsf{wt}(s_P).$$

The inclusive interval in Equation (1) is therefore equivalent to $110 \le \mathsf{wt}(s_P) \le 150$. Summing the corresponding binomial coefficients gives

$$p = 2^{-242} \sum_{j=110}^{150} \binom{242}{j} = 0.9303737782253141.$$

The attacker hashes distinct fresh messages and stops at the first accepted completion. This is a geometric experiment with expectation $1/p$. It never calls the signing oracle or uses a secret key. □

The method is polynomial-time public linear algebra. It does not solve a generic exponential low-weight syndrome-decoding problem. The broad central weight interval makes one fixed affine representative acceptable with constant probability.

3.2 Claim boundary

This theorem is existential: the attacker selects its successful message by public search. It does not establish a universal signature, success for every fixed preselected message, or an unconditional property of a particular concrete hash function. A fixed SHAKE256 run is an executable instance of the paper-level attack, while the exact probability is a random-oracle statement.

4 Full-parameter evidence

4.1 Primary reconstruction

The sealed primary Python implementation independently reconstructed the printed Gen1/Gen2 algebra at complete SSD1760 dimensions. Its valid structured key had rank 242 for both public matrices. An honest weight-499 signature passed the ordinary verifier. The public attack fixed 363 free ones, obtained pivot weight 116 on its first counter message, and produced an accepted weight-479 response with zero signing queries.

The ordinary verifier rejected the response after changing the message, changing one response bit while retaining an in-range weight, altering the public key, and replacing the response by zero. A separately sealed control reconstructed the same public key and rejected weights 0 and 1760. The full primary process took 4.87 seconds wall time, 19,968 KiB peak resident memory, and one worker. The primary result has SHA-256

`87a78f23b8189f27f2339f56a871b4e3871c5128a6f8551953eddaf49b22089d.`

4.2 Hostile adversarial reconstruction

The clean adversarial implementation was written independently in Magma from the pinned paper and modeled one fresh uniform 1760-bit oracle response for each distinct message. It used the paper's consecutive odd signing exponents $[11, 13, 15, 17]$, obtained a valid rank-242 key on its first Gen1 attempt, and accepted an honest weight-501 signature.

The public solve tracked the complete row transform, verified the pivot identity, and agreed with a separate inverse of the original pivot-column submatrix. Its first candidate had free, pivot, and total weights 363, 110, and 473 and passed Algorithm 4. Changed-message, in-range one-bit, and independent structured wrong-key controls rejected.

The adversarial run also constructed responses that satisfied the public equation at each neighboring endpoint. Table 1 isolates the inclusive weight check.

Response weight	Public equation	Expected	Ordinary verifier
472	holds	reject	reject
473	holds	accept	accept
513	holds	accept	accept
514	holds	reject	reject

Table 1: Equation-valid boundary controls for Algorithm 4.

The clean source was sealed before execution at SHA-256

`145d28c510234e76459598b917dc4c55aade5156a68038f019d1196685b431b1.`

The run exited zero with empty stderr and no failed assertion. Its attack stage took 0.19 CPU seconds. The complete process took 1.29 seconds wall time and 23,552 KiB peak resident memory on one worker. An earlier provisional run that emitted an interpreter error is explicitly superseded and contributes no fact or measurement here.

4.3 Fresh-context independent reproduction

A separately tasked reproducer worked from the pinned paper and attack description without reading the primary or adversarial implementations or results. It independently reconstructed the full standard SSD1760 key, signing, verification, and public-completion algorithms in Python. Five fresh full-row-rank structured keys were generated from operating-system entropy, and all five yielded accepted fresh-message forgeries under the unchanged ordinary verifier.

The central public-hash query counts were $1, 2, 1, 1, 1$, six in total, with zero signing queries. Forgery weights were $482, 489, 489, 478, 478$. Attack times from immediately before public row reduction were 0.052984–0.053914 seconds per key and 0.267922 seconds total. The whole five-key process took 14.76

seconds wall time, 10.66 seconds user CPU, 0.07 seconds system CPU, and 24,576 KiB peak resident memory on one worker.

For every key, a separate honest signature accepted; changed-message, weight-preserving changed-signature, and independent wrong-key controls rejected; and equation-valid weights 472, 473, 513, and 514 respectively rejected, accepted, accepted, and rejected. A separately sealed validator then rechecked only the retained public keys, messages, signatures, hashes, equations, weights, freshness, and query counts and reported a pass. The sealed independent results have SHA-256

`9b9843dfca266117d3d0a098953816feabb9ce1b19e225cadc79ded93989a878.`

The 30-entry experiment manifest has SHA-256

`2608c672de46fd87302c8a8b6d5f3c08b3fb1ffbde39be599e4c8a7facf1abb6.`

4.4 Public package reproducer

The polished Python reproducer uses only the standard library. It builds two full structured keys, checks honest acceptance, executes the public attack, and repeats the dual RREF/pivot-inverse cross-check. It uses SHAKE256 on explicit distinct message encodings and reports public message and signing query counts separately. Its controls include a changed message, an in-range response mutation, an independent wrong key, zero response, and four equation-valid boundary responses.

The package additionally verifies the separate Gen1 rank defect described below. Ordinary commands draw fresh operating-system seeds. Disclosed fixed seeds produce deterministic reference JSON. Timing values are excluded from those committed JSON files; external measurements must identify either the attack-stage or complete-process boundary.

5 Specification issues

These issues are relevant to interpretation but are not the cause of the forgery.

Gen1 rank defect. The paper permits $\alpha_0 = r = 15$. With $Q = I + M^r$, the Gen1 middle factor then becomes

$$I + M^{\alpha_0} + R_0 Q = Q + R_0 Q = (I + R_0)Q.$$

Its rank is at most $\operatorname{rank}(Q) = 198$, below the 242 independent rows Gen2 requests. Left multiplication cannot increase the rank. Both sealed attack runs used valid full-rank keys with other α_0 values; the public forgery relies only on those valid keys.

Signing-threshold ambiguity. Algorithm 3 prints a while-condition containing both $\mathsf{wt}(u') < \omega - \mu$ and $\mathsf{wt}(u') < d - \mu$, while the surrounding prose discusses the $\omega - \mu$ lower bound. Because $d > \omega$, the literal condition imposes the higher value 491. The sealed honest weights 499 and 501 satisfy both readings, and Algorithm 4 unambiguously accepts from 473 through 513.

Missing concrete interface. The paper defines a 1760-bit random oracle but no concrete hash, message encoding, signature serialization, or author implementation. The attack is presented at the ordinary mathematical interface. The executable SHAKE256 adapter is explicit and is not described as an author wire format.

SSD2744 inconsistency. Table I prints $k' = 377$ for SSD2744, while Tables III and VI print $k' = 337$. This report makes no claim for SSD2744.

6 Timer semantics

The reliable adversarial attack timer starts immediately before public row reduction and stops after the first candidate passes ordinary verification. It reports 0.19 *CPU seconds*. The 1.29-second *wall time* covers

a larger process: first structured key generation, honest conformance, the attack, a second structured wrong key, and endpoint controls. Its peak memory was 23,552 KiB.

The primary JSON field named `seconds_including_rref` is mislabeled. Its timer begins after row reduction and the recorded 0.000544 seconds covers only candidate-message completion. We do not cite it as an attack-stage or complete-process runtime. CPU and wall measurements, and their different scopes, are kept separate throughout this package.

7 Attack history and repair direction

The mandatory final exact-title, identifier, author, mechanism, citation, repository, and public-discussion recheck through 14 October 2026 found the proposal, bibliographic and resource pages, an ordinary downstream citation, the distinct 2023/1956 FDSD successor [3], and unrelated syndrome-decoding work. It located no previous public exact-target key recovery, equivalent signer or witness recovery, practical forgery, substantive parameter compromise, or application of this affine-completion mechanism to SSD. This moderate-high-confidence result is a bounded public search and cannot exclude private, deleted, unpublished, or unindexed work. We make no priority claim and no novelty claim for the underlying linear algebra.

The ordinary verifier needs a relation whose public affine solution space does not intersect its accepted response distribution with constant probability. A repair must reconsider the public dimensions, response constraint, and security reduction together, then publish a complete hash and encoding profile and repeat correctness and full-parameter analysis. Merely rejecting one fixed free assignment has not been shown sufficient. This report does not validate a drop-in patch.

8 Reproducibility and publication status

The public package contains a landing report, machine-readable metadata, an exact affected-row record, full source for the standard-library reproducer, deterministic reference outputs, this LaTeX source and PDF, and a SHA-256 manifest. These materials are published in the ePrint Signature Audit repository.

Primary reproduction, hostile adversarial review, fresh-context independent reproduction on five new full-rank SSD1760 keys, final exact-target history review, and independent Publication Reviewer review all passed. The package is published and classified as `VERIFIED_ATTACK`.

Acknowledgements

The computations were performed on the Norwegian Research and Education Cloud (NREC), using resources provided by the University of Bergen and the University of Oslo.

References

[1] Abdelhaliem Babiker. Digital Signature from Syndrome Decoding Problem. IACR Cryptology ePrint Archive, Report 2022/1698, 2022. https://eprint.iacr.org/2022/1698.

[2] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A Digital Signature Scheme Secure against Adaptive Chosen-Message Attacks. *SIAM Journal on Computing*, 17(2):281–308, 1988.

[3] Abdelhaliem Babiker. A Signature Scheme from Full-Distance Syndrome Decoding. IACR Cryptology ePrint Archive, Report 2023/1956, 2023. https://eprint.iacr.org/2023/1956.